

Eco-design Digitale di Base per i servizi ICT

Programmazione in C e Python

Obiettivi della lezione

- Capire come il processo di import dei vari moduli
- Introdurre lo scripting per automatizzare attività comuni
- Introdurre file JSON
- Vari esempi di scripting
- Realizzare semplici analisi su file di testo o CSV

Script Python

- File di testo con codice Python
- Si esegue con `python3 nomefile.py`
- Utile per automatizzare operazioni ripetitive

Perché usarli?

- Automatizzare backup, lettura/scrittura file, invio email...
- Semplificare task ripetitivi
- Rendere il lavoro più efficiente

Moduli Python

- File .py con funzioni, classi, costanti
- Si importano per riutilizzare codice
- Esistono moduli **built-in**, **esterni** e **personalizzati**

Moduli Python

Processo di import

1. Parsing dell'istruzione import

Quando l'interprete incontra una riga come:

```
import mio_modulo
```

Il parser la riconosce come un'**istruzione di importazione** e chiama internamente:

```
__import__('mio_modulo')
```

Moduli Python

Processo di import

2. Controllo nella cache (sys.modules)

- Python mantiene una **cache globale** dei moduli già caricati:
- Se mio_modulo è già presente in sys.modules, Python **restituisce subito il riferimento**, evitando di ricaricarlo.
- **Vantaggio:** efficienza e coerenza (tutti i moduli condividono lo stesso oggetto in memoria)

Moduli Python

Processo di import

3. Risoluzione del nome del modulo

- Se non è nella cache, Python cerca di **risolvere il nome** del modulo con il meccanismo dei namespace:
- Se `import mio_modulo.submod`, Python cerca prima `mio_modulo`, poi `submod` all'interno.
- Python separa i nomi da sinistra verso destra e cerca di costruire l'oggetto finale.

Moduli Python

Processo di import

4. Ricerca fisica del modulo

- Python usa una **lista di percorsi** per cercare i file corrispondenti al modulo:
 - La directory dello script chiamante
 - PYTHONPATH (variabile d'ambiente)
 - Le directory predefinite (es. site-packages)
 - Zip archive o pacchetti namespace

Moduli Python

Processo di import

5. Loader e Finder

Python segue il protocollo **import machinery**, composto da due parti principali:

- Finder
- Loader

Moduli Python

Processo di import

6. Compilazione in bytecode

- Se il modulo .py viene letto per la prima volta:
- Python **compila** il file in bytecode (.pyc)
- Lo salva nella cartella `__pycache__`/
- Se è già presente un .pyc valido, viene **riusato**

Moduli Python

Processo di import

7. Esecuzione del modulo

- Python **esegue tutto il codice** contenuto nel file .py **una sola volta**
- Tutte le funzioni, classi e variabili definite diventano attributi del modulo

Moduli Python

Processo di import

8. Il modulo viene inserito in `sys.modules`

9. Reload del modulo (in caso di modifica del modulo)

Script e Automazione

- Gestione automatica dei file
- Interazione con servizi web
- Gestione informazioni
- Elaborazione grandi quantità di dati

Script e Automazione

- Gestione automatica dei file
- Interazione con servizi web
- Gestione informazioni
- Elaborazione grandi quantità di dati

JSON

- **JavaScript Object Notation**
- Usato per rappresentare dati strutturati
- Facilmente leggibile sia da umani che da computer

```
{
```

```
  "nome": "Massimo",
```

```
  "età": 25,
```

```
  "linguaggi": ["Python", "C", "Java"]
```

```
}
```

JSON in python

- Utilizzo del modulo json
- Interazione tra oggetti JSON e variabili python molto semplice

Quarto esercizio

Scrivi un programma che:

Chieda all'utente quanti numeri desidera inserire.

Per ciascun numero:

1. Se è positivo e pari, stampa "Positivo e pari".
2. Se è positivo e dispari, stampa "Positivo e dispari".
3. Se è zero, stampa "È zero".
4. Se è negativo, stampa "Numero negativo".

Al termine, stampa:

Il conteggio dei numeri positivi, negativi e pari.
Il totale e la media dei numeri inseriti.